Gardener F5 Application Load Balancer Extension

Part 1 - Architecture Basis, Capability Mapping, and Target Solution

Target design baseline
Greenfield architecture derived from AWS controller patterns and constrained by verified CMP capabilities

Not an assessment of the current implementation

1. Purpose and design method

This part establishes the architectural basis and the target solution for a Gardener extension that provides application load balancing through CMP/F5 for Shoot clusters.

The design uses three inputs with deliberately different authority:

- AWS Load Balancer Controller repository - architectural reference for controller decomposition, model building, resource graphs, deployers, finalizers, event fan-out, ownership tracking, and idempotent reconciliation.
- CMP Swagger/OpenAPI - authoritative capability contract for what CMP can provision on F5.
- Existing Gardener F5 extension repository - evidence of CMP endpoint structure, authentication headers, request transport, and response handling; not the architectural baseline.

The resulting architecture is therefore not a copy of AWS and not a rewrite of the current extension. It is a Gardener-specific controller architecture that adopts the AWS patterns that remain valid when mapped to CMP.

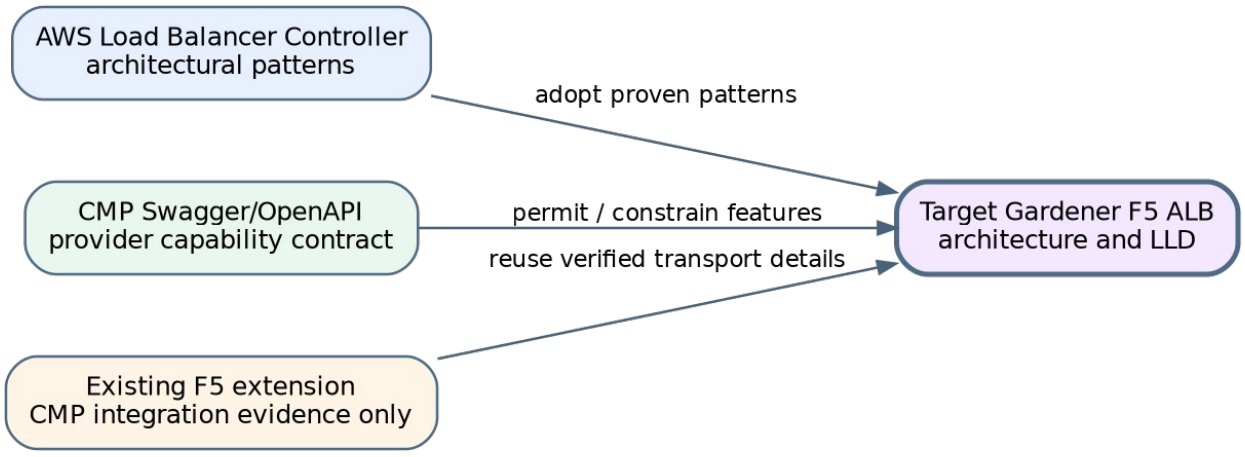

Figure 1 - Design inputs and authority

1.1 Governing decision rule

```
Kubernetes requirement
    -> AWS implementation pattern
    -> CMP capability verification
    -> F5 extension decision

CMP fully supports it    -> implement
CMP supports a subset    -> implement the supported subset and validate explicitly
CMP does not support it  -> reject or defer; never silently approximate
```

2. Repository-derived architectural findings

2.1 Patterns adopted from AWS Load Balancer Controller

The supplied AWS repository shows the same high-level flow in both Service and Ingress controllers: build an in-memory model, deploy the model, manage finalizers, and publish status. Ingress uses a group reconciliation key and deployment is delegated to resource-specific managers.

- Separate Service and Ingress reconciliation entry points.
- Thin controllers that orchestrate fetch, model build, deploy, finalizer, status, events, and retry.
- Resource-specific model builders.
- An in-memory resource stack with logical resources and dependencies.
- Resource-specific deployment managers rather than provider CRUD in controllers.
- Ingress group reconciliation for safe shared frontends.
- Dependent-resource event mapping back to the owning reconciliation key.
- Explicit tracking/ownership and finalizer-based cleanup.

Repository evidence: controllers/service/service_controller.go; controllers/ingress/group_controller.go; pkg/service/model_builder.go; pkg/ingress/model_builder.go; pkg/model/core; pkg/deploy/elbv2.

2.2 CMP capabilities verified from Swagger

The supplied CMP Swagger exposes a decomposed resource API that can support an AWS-inspired desired-state graph without importing AWS-specific resource semantics.

Capability	Verified CMP operation
LB Service	create/list/get/delete; flavor, network, VPC, labels, optional HA
VIP	allocate/list/delete under LB Service
Virtual Server	create/list/get/patch/delete; protocol, port, algorithm, persistence, redirect, XFF
Pools	create/get/delete; multiple pools per Virtual Server; set default
Members	add/update/delete; health and operating-state operations
Monitors	create/list/update/delete and availability policy
Routing rules	host, path, match type, target pool
Certificates	upload/list/delete and attach to Virtual Server
Backend identity	network port search by IP and compute/network APIs
Ownership lookup	labels and label-based LB lookup

2.3 Existing extension evidence retained

Only provider-integration evidence is retained: organisation/project-scoped paths, Ce-Auth/HMAC and scoped headers, redirect suppression, TLS handling, request ID/error classification, Retry-After handling, rate limiting, and practical LB Service/VIP/Virtual Server calls. Raw JSON does not cross the new typed-client boundary.

3. Target system boundary

The application load-balancer controller is a managed Shoot component delivered by the Gardener extension lifecycle. The lifecycle actuator installs, upgrades, configures, and removes the controller. The application controller then reconciles Service and Ingress resources inside the Shoot.

Plane	Responsibility
Extension lifecycle plane	Deploy controller, provide configuration and credentials, upgrade and remove it.
Application load-balancing plane	Translate Shoot Service/Ingress state into CMP/F5 desired state and maintain it continuously.

4. Target architecture

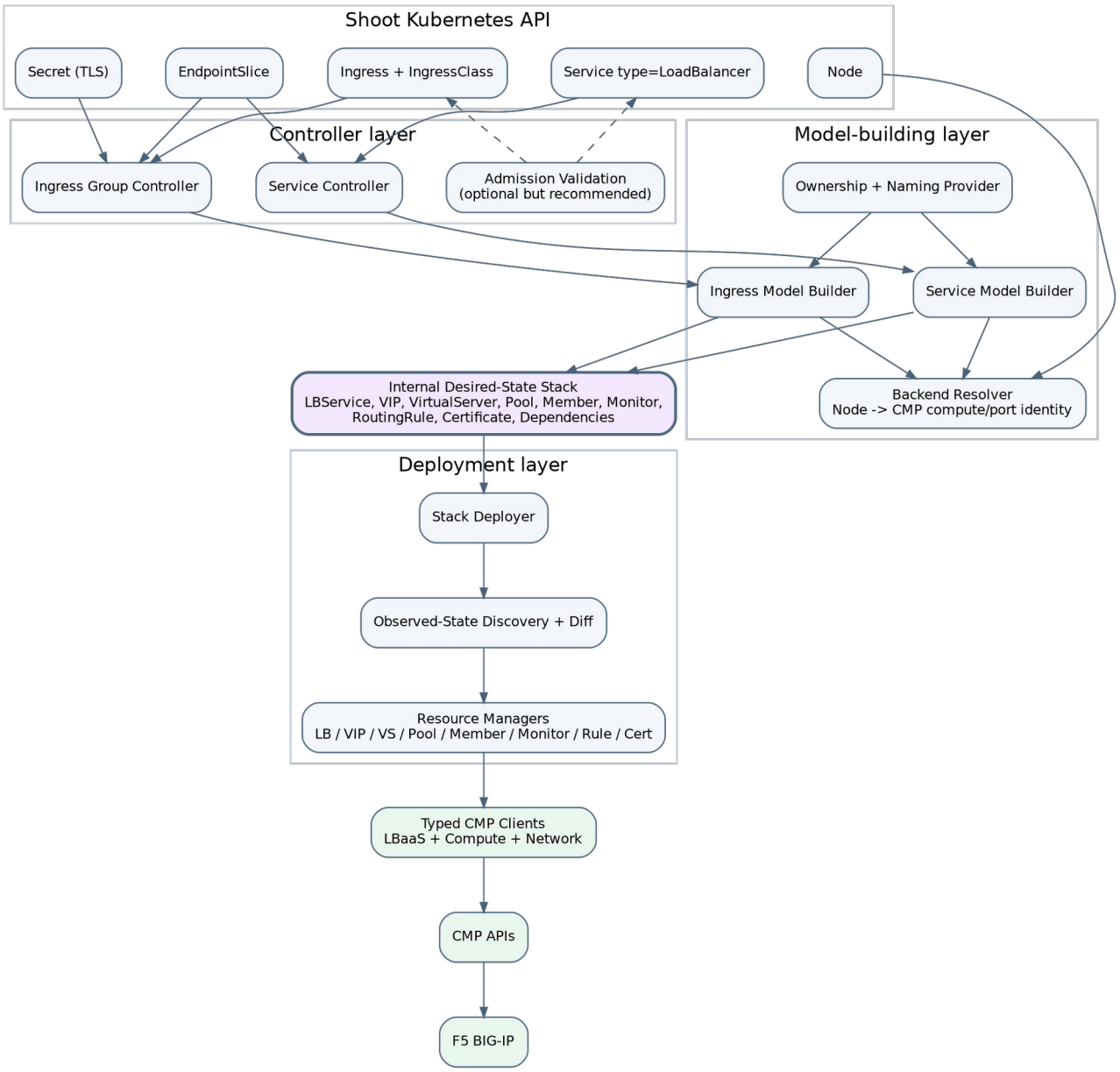

Figure 2 - Target component architecture

4.1 Controller layer

A Service Controller handles selected type=LoadBalancer Services. An Ingress Group Controller handles Ingresses selected by the F5 IngressClass. Controllers load objects, manage finalizers, call builders and deployers, publish status, and classify retries. They do not construct CMP requests.

4.2 Model-building layer

Service and Ingress builders convert Kubernetes resources and provider configuration into one typed internal Stack. They resolve backends through interfaces, validate feature combinations, generate logical IDs and ownership, and never mutate CMP.

4.3 Internal desired-state stack

The Stack contains typed LBService, VIP, VirtualServer, Pool, BackendMember, HealthMonitor, RoutingRule, and Certificate resources with stable logical IDs and references. It is rebuilt each reconciliation and is not stored in etcd.

4.4 Deployment layer

A Stack Deployer coordinates resource managers. Each manager discovers owned actual resources, normalizes desired and observed values, computes minimal changes, applies create/update/replace/delete operations, and returns observed readiness.

4.5 Typed CMP clients

LBaaS, Compute, and Network clients centralize transport, authentication, scoped headers, timeouts, rate limiting, error classification, request IDs, and typed encoding/decoding.

5. CMP/F5 resource model

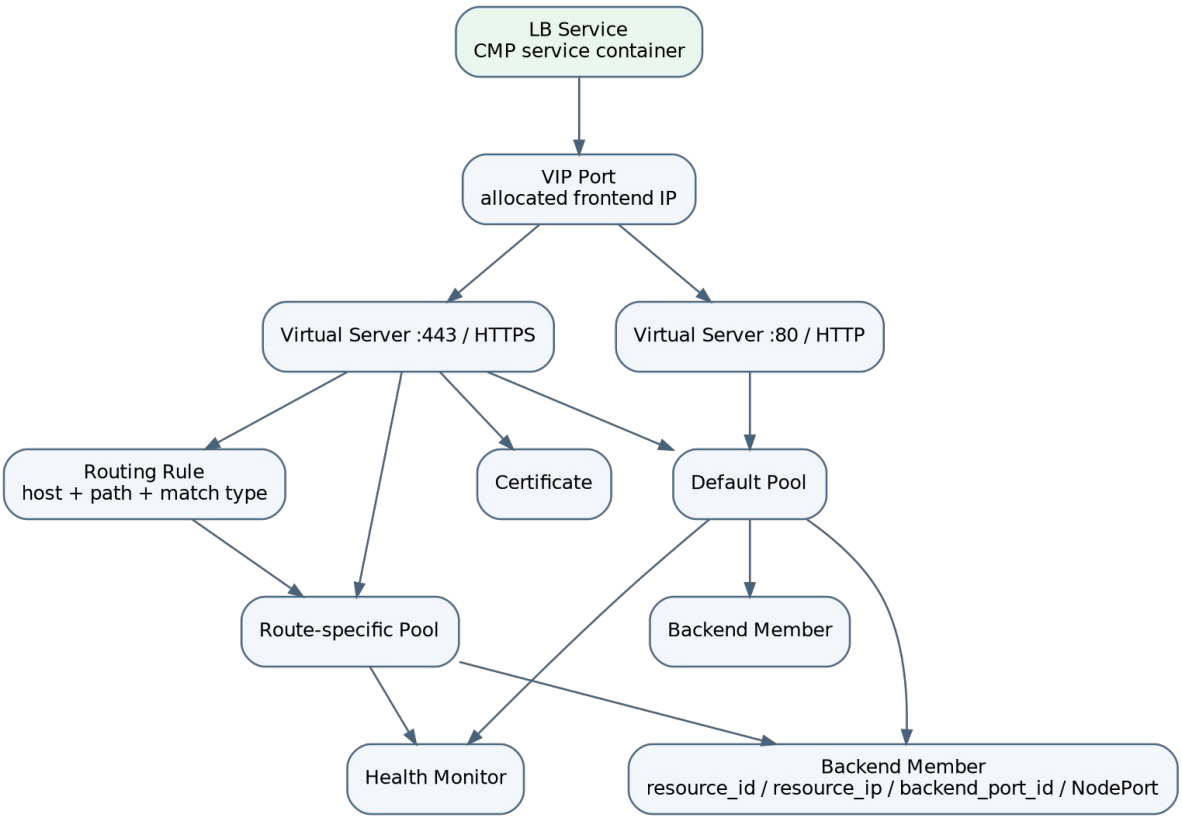

Figure 3 - CMP/F5 resource graph for application load balancing

5.1 Why the LB Service is mandatory

The LB Service is the parent CMP resource for both Service and Ingress flows. It owns the VIP and Virtual Servers. A routing-only diagram that starts at a host name omits the provider container and must not be treated as the actual resource hierarchy.

```
Service -> LB Service -> VIP -> Virtual Server(s) -> Pool(s) -> Members
Ingress Group -> LB Service -> VIP -> HTTP/HTTPS Virtual Server(s)
                              -> Pools + Routing Rules + Certificates
```

5.2 Logical relationship rules

- Initial release uses one VIP per Service stack or Ingress group stack.
- Multiple Virtual Servers may share the VIP when protocol/port tuples are distinct.
- A Virtual Server has one default pool and may have additional route-specific pools.
- Routing rules select pools by host/path/match type.
- Pools own members and health monitors.
- Certificates are uploaded under the LB Service and attached to HTTPS Virtual Servers.

6. Kubernetes-to-CMP mapping

6.1 Service type LoadBalancer

Kubernetes input	Internal model	CMP/F5 resource
Service UID	stack ownership root	labels/name on all resources
Service port	frontend	Virtual Server
Service protocol	frontend protocol	Virtual Server protocol
Service NodePort	target port	member port
eligible Node	backend target	Pool Member
Node InternalIP	target address	resource_ip/address
CMP compute identity	provider target identity	resource_id/compute ID
CMP port found by IP	network identity	backend_port_id/port ID
health configuration	monitor spec	Health Monitor
allocated VIP	observed frontend	Service status

6.2 Ingress

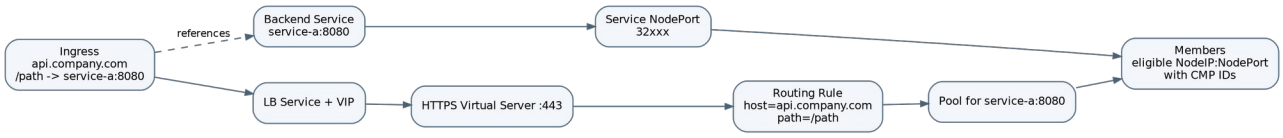

Figure 4 - Ingress to CMP/F5 mapping (including LB Service)

Kubernetes input	Internal model	CMP/F5 resource
Ingress group	shared stack root	LB Service + VIP
HTTP/HTTPS listener	frontend	Virtual Server
host/path	route	Routing Rule
backend Service/port	backend group	Pool
default backend	default route	default Pool
TLS Secret	certificate model	CMP Certificate + attachment
eligible Nodes	targets	Pool Members

6.3 Backend resolution

The initial target mode is NodeIP:NodePort. EndpointSlices determine which nodes actually host ready endpoints. Nodes are then mapped to CMP compute/network identity, including backend port ID. The controller never invents provider IDs.

```
EndpointSlice ready endpoints
 -> nodeName set
```

```
-> Node readiness and traffic-policy filter
-> Node InternalIP + Service NodePort
-> CMP port search by IP + compute/network lookup
-> BackendMember(resource_id, resource_ip, backend_port_id, port, weight)
```

7. Capability matrix and target decisions

Capability	AWS pattern	CMP evidence	Decision
Service type LoadBalancer	service controller + model builder	LB Service/VIP/VS/pools/members	Implement
Multi-port Service	multiple listeners/target groups	multiple VS under one LB Service	Implement one VS/pool per port
Ingress host routing	listener rules	host/path routing rule	Implement
Ingress path routing	listener rules	path + match_type	Implement supported match types
Ingress groups/shared VIP	group reconciler	one LB/VIP with many pools/rules	Implement with compatibility validation
Default backend	default action	set-default pool	Implement
TLS termination	certificate/listener model	certificate upload + attach	Implement
HTTP->HTTPS redirect	redirect action	redirect_https	Implement
X-Forwarded-For	listener attributes	x_forwarded_for	Implement
Persistence	attributes	persistence and cookie fields	Implement validated subset
Health checks	target-group health checks	monitor CRUD	Implement
Weighted members	target configuration	partially evident; payload verification needed	Conditional
Source CIDRs	listener inbound CIDRs	only coarse API clearly exposes allowed_cidrs	Defer pending decomposed endpoint
Pod-IP targets	IP target mode	reachability/identity not proven	Not initial release
Dual stack	IP address type	not established	Not initial release
Multi-AZ/HA	subnet/AZ model	LB Service ha flag; platform support gated	Architecture-ready, release-gated
Gateway API	gateway model	primitives compatible	Future

7.1 Validation behavior

The model builder is the authoritative validator because it has full dependency context. A validating webhook is recommended for static errors such as malformed annotations, unsupported protocols/path types, shared-group conflicts, listener collisions, missing TLS references, and invalid persistence combinations. Flavor availability and backend identity remain reconciliation-time checks.

8. Ownership, identity, and adoption

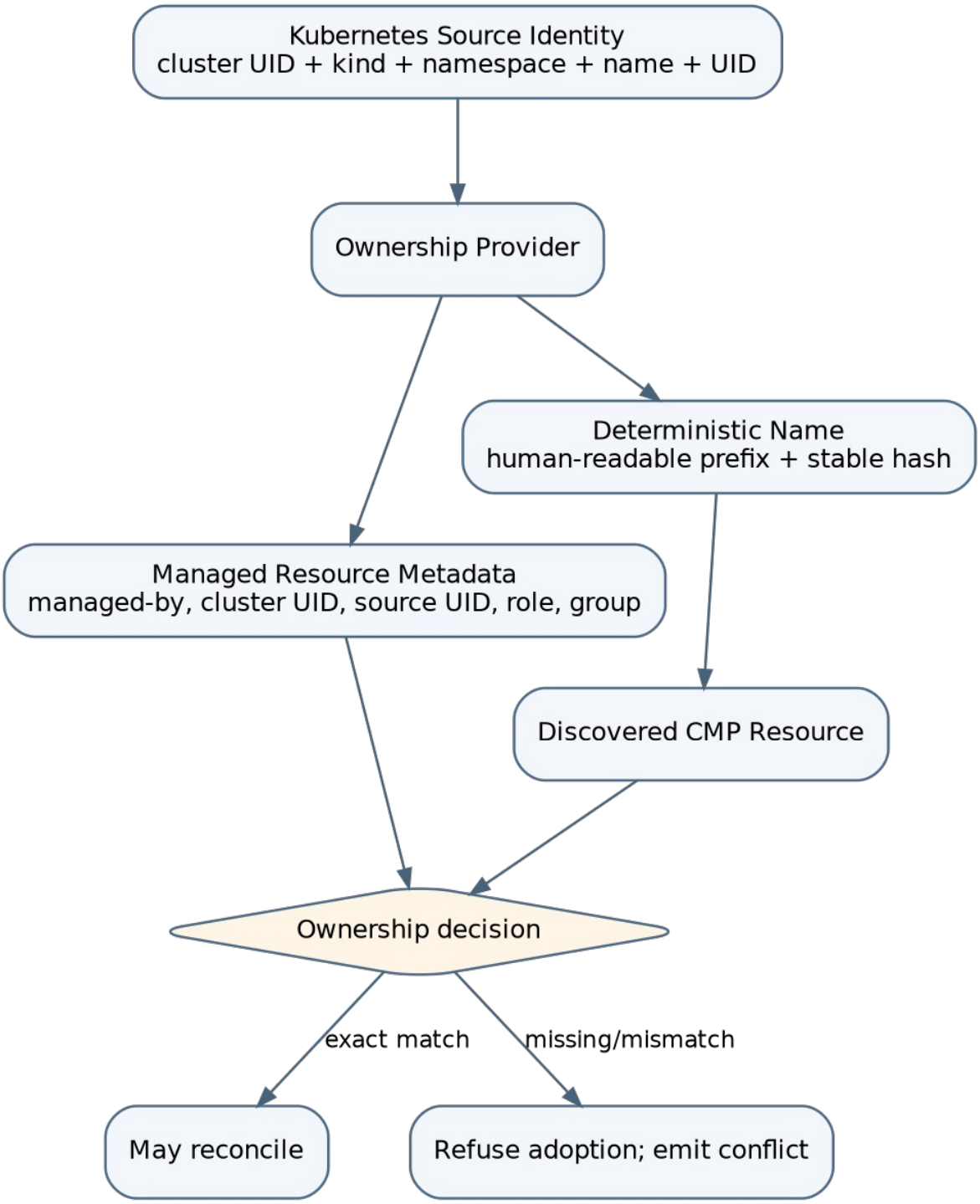

Figure 5 - Ownership generation and adoption decision

8.1 Who creates ownership fields

The Ownership Provider in the model-building layer creates ownership fields. Users do not supply them, controllers do not improvise them, and the admission webhook only validates user configuration.

```
managed-by      = gardener-extension-f5
cluster-uid     = <Shoot/cluster stable UID>
source-kind     = Service | IngressGroup
source-namespace = <namespace or group scope>
source-name     = <object name or group key>
source-uid      = <Kubernetes UID or deterministic group UID>
resource-role   = lb-service | vip | virtual-server | pool | member | monitor | rule | certificate
logical-id      = <stable internal logical ID>
```

8.2 Adoption and finalizers

Adoption requires matching ownership, tenant/project, VPC/network, resource role, and logical ID. A matching name alone is insufficient. Finalizers are added before provider creation and removed only after owned resources or shared bindings are cleaned up.

9. Reconciliation algorithms

9.1 Service reconciliation

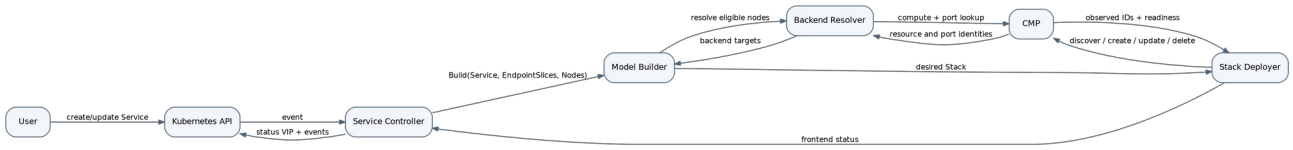

Figure 6 - Service reconciliation sequence

1. Read and classify the Service.

2. Handle unselected or deleting state safely.

3. Resolve ports and NodePorts.

4. Read EndpointSlices and derive nodes with ready endpoints.

5. Apply Node readiness and externalTrafficPolicy filtering.

6. Resolve CMP compute/network identity and backend port ID.

7. Build the desired Stack and ownership metadata.

8. Add finalizer before provider mutation.

9. Deploy resources in dependency order.

10. Publish VIP only after required frontend readiness.

11. Emit Conditions, Events, logs, and metrics.

9.2 Ingress group reconciliation

1. Resolve group key and load active/inactive members.

2. Validate group-wide network, VIP, listener, and ownership compatibility.

3. Load backend Services, EndpointSlices, Nodes, and TLS Secrets.

4. Deduplicate pools by backend Service/port and policy tuple.

5. Build LB Service, VIP, Virtual Servers, pools, monitors, members, rules, and certificates.

6. Add finalizers to active members.

7. Deploy the group Stack.

8. Update all active Ingress statuses with the shared VIP.

9. Remove inactive-member finalizers after their bindings/routes are absent.

9.3 Dependency order and stable frontend rule

```
Create/update:
LB Service -> VIP -> Certificate -> Virtual Server -> Pool -> Monitor -> Member -> Routing Rule -> set default pool

Delete:
reverse dependency order
```

Backend-only changes must not recreate the LB Service, VIP, Virtual Server, routing rules, certificates, or unchanged pools. Each manager owns equality and mutation for its resource type.

10. Package and interface blueprint

```
cmd/gardener-extension-f5/          # extension lifecycle and component registration
cmd/f5-application-lb-controller/  # Shoot controller process
pkg/controllers/service/
pkg/controllers/ingress/
pkg/model/core/
pkg/model/f5/
pkg/service/
pkg/ingress/
pkg/backend/
pkg/ownership/
pkg/deploy/
pkg/cmp/lbaas/
pkg/cmp/compute/
pkg/cmp/network/
pkg/status/
pkg/finalizers/
pkg/validation/
pkg/webhooks/
pkg/metrics/
```

Representative contracts:

```
type ServiceModelBuilder interface {
   Build(ctx context.Context, svc *corev1.Service) (*core.Stack, error)
}

type IngressModelBuilder interface {
   Build(ctx context.Context, group ingress.Group) (*core.Stack, error)
}

type StackDeployer interface {
```

```
    Deploy(ctx context.Context, stack *core.Stack) (*ObservedStack, error)
}

type BackendResolver interface {
    ResolveNode(ctx context.Context, node *corev1.Node) (ProviderTarget, error)
}
```

11. Error and retry contract

Error category	Controller behavior
Permanent configuration	No rapid retry; wait for Kubernetes change
Dependency not ready	Bounded RequeueAfter
Asynchronous CMP operation	Poll with bounded interval and timeout state
HTTP 429	Honor Retry-After
HTTP 401/403	Condition/Event and slow retry
Transport/5xx	controller-runtime backoff
404 during delete	Treat as success
Ownership conflict	Permanent until operator resolves collision
Lost create response	Rediscover by ownership/logical ID before another create

12. Architecture decisions fixed by Part 1

1. The application LB is a dedicated Shoot controller installed by the Gardener extension lifecycle.

2. Service and Ingress use separate thin controllers and shared deployment infrastructure.

3. Ingress uses a group abstraction even for a one-member group.

4. Every reconciliation builds a complete internal desired-state Stack before mutation.

5. CMP mutation occurs only through resource-specific managers and typed clients.

6. The decomposed LB Service/VIP/Virtual Server/Pool/Member API is the primary provider model.

7. Ingress always includes an LB Service in the provider hierarchy.

8. Initial backend mode is NodeIP:NodePort with CMP identity and backend-port resolution.

9. Provider resource IDs are discovered, never guessed.

10. Ownership is generated internally for every logical resource.

11. Deterministic names supplement but never replace ownership proof.

12. Static admission validation and provider-dependent reconciliation validation are separated.

13. Backend-only changes do not recreate stable frontend resources.

14. Unsupported capabilities are explicitly rejected or deferred.

13. Source evidence used

13.1 AWS repository

controllers/service/service_controller.go; controllers/ingress/group_controller.go; controllers/*/eventhandlers; pkg/service/model_builder.go; pkg/ingress/model_builder.go; pkg/model/core; pkg/deploy/elbv2 managers.

13.2 CMP Swagger

compute-swagger.json load-balancer, LB Service, VIP, Virtual Server, pool, member, monitor, routing-rule, certificate, and network-port endpoints; network-manager.json network/subnet context.

13.3 Existing Gardener F5 repository

pkg/cmp/lbaas/client.go; pkg/f5/client.go; pkg/controller/lifecycle/extension_controller.go; pkg/apis/f5/v1alpha1/types.go. Used only for provider integration evidence.

13.4 Previous LLD

Useful requirements were retained and refined: thin controllers, builders, desired/observed separation, managers, backend resolution, ownership, finalizers, status, idempotency, observability, security, package design, and testing direction. Unverified assumptions were not carried forward.

14. Scope of Part 2

Part 2 will define the full internal desired-state model and low-level contracts: every resource struct, logical ID, dependency, normalization rule, ownership field, equality/diff rule, immutable-field replacement rule, and observed-state representation.